



MULTICLOUD DEVOPS BY VEERA SIR

AWS THREE-TIER HIGH-AVAILABILITY & MULTI-REGION DISASTER RECOVERY ARCHITECTURE

Project Explanation & Interview Preparation Guide

multicloud-devops.veerasir

Prepared for technical interview walkthroughs and project documentation

Table of Contents

Table of Contents	2
1. Project Overview	3
2. Architecture Diagram — Multi-Region HA & Disaster Recovery	3
2.1 How Traffic Flows	4
2.2 High Availability Inside a Region	4
2.3 Disaster Recovery Strategy — Warm Standby	4
2.4 Centralized Monitoring & Logging	4
3. Architecture Diagram — Detailed VPC & Three-Tier Network Design	5
3.1 Network Layout	5
3.2 Tier-by-Tier Breakdown	5
3.3 Security Group Chain (Defense in Depth)	6
4. AWS Services Used — Quick Reference	6
5. Interview Q&A — Anticipated Deep-Dive Questions	7
6. Full Project Explanation — Interview Narrative (Paragraph Form)	9
7. Presentation Tips for the Interview	10

VEERA

1. Project Overview

This project is a production-grade, highly available, and disaster-recovery-ready three-tier web application architecture built entirely on AWS. It demonstrates end-to-end cloud infrastructure design covering networking, compute, database, security, backup, and observability — the same patterns used by real companies to run mission-critical applications.

The architecture is split into two complementary designs that together tell the full story of the project:

- A multi-region Active/Warm-Standby design (us-east-1 as Primary, us-west-2 as DR) that focuses on business continuity, cross-region backup/replication, DNS failover, and centralized monitoring.
- A single-region, multi-AZ VPC design that drills into the detailed network layout — public/private subnet separation, security group traffic flow, load balancer tiers, and data-tier replication (RDS Multi-AZ + ElastiCache).

Together, these diagrams cover the two things interviewers most often probe: 'How is your application highly available inside one region?' and 'What happens if the whole region goes down?'

2. Architecture Diagram — Multi-Region HA & Disaster Recovery

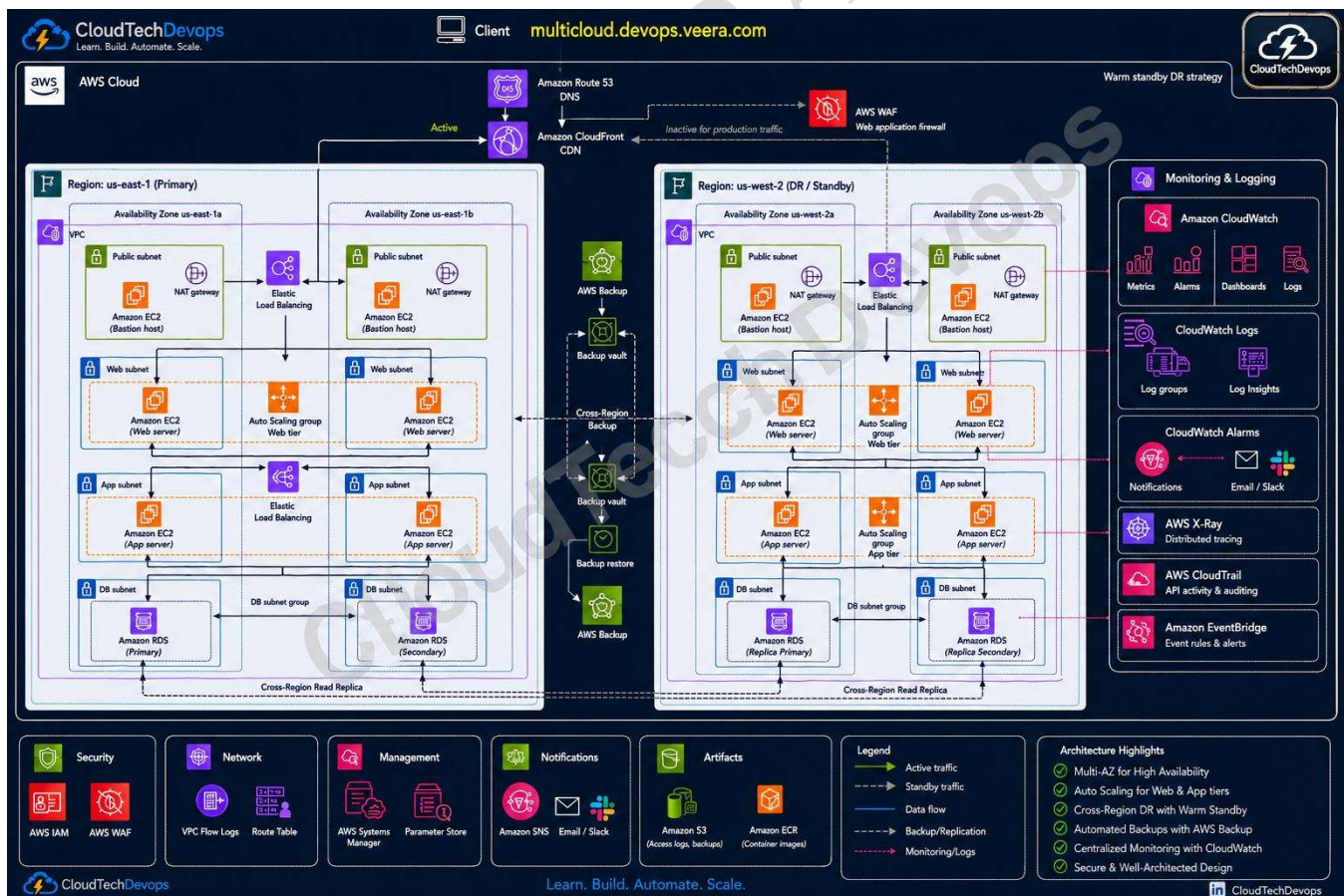


Figure 1: Warm-standby DR architecture across us-east-1 (Primary) and us-west-2 (DR), with centralized monitoring and backup.

2.1 How Traffic Flows

- A client requests multicloud-devops.veerasir, which resolves through Amazon Route 53 (DNS).
- Route 53 uses a health-check-based failover routing policy: normally it points to Amazon CloudFront (CDN) which is 'Active' and serves the Primary region (us-east-1).
- AWS WAF sits in front to filter malicious traffic (SQLi, XSS, rate-based rules) and is 'Inactive' for production traffic until failover is triggered, keeping the DR path cost-optimized until it's actually needed.
- Inside each region, traffic lands on a public Elastic Load Balancer, passes through Bastion Hosts (for admin SSH only, never for application traffic), then into the Web tier, App tier, and finally the DB tier — each tier isolated in its own subnet and Auto Scaling Group where applicable.

2.2 High Availability Inside a Region

- Every tier (Web, App, DB) is deployed across two Availability Zones (us-east-1a and us-east-1b) so the loss of a single AZ doesn't take down the application.
- Auto Scaling Groups on the Web and App tiers automatically replace unhealthy instances and scale out under load.
- Amazon RDS runs as Primary/Secondary within the region for synchronous Multi-AZ failover, in addition to being the source for cross-region read replicas.

2.3 Disaster Recovery Strategy — Warm Standby

The DR region (us-west-2) is a scaled-down mirror of production: the same subnet layout, the same tiers, and Auto Scaling Groups configured but typically running at a lower capacity. This is a 'warm standby' pattern — a middle ground between cheap-but-slow (backup/restore) and expensive-but-instant (active-active):

- Cross-Region Read Replica: Amazon RDS in us-west-2 continuously replicates from the Primary RDS in us-east-1, so the DR database is always close to current.
- Cross-Region Backup: AWS Backup takes scheduled snapshots into a Backup Vault in the Primary region and copies them to a second vault, giving a point-in-time restore path independent of live replication.
- Failover: if us-east-1 becomes unhealthy, Route 53 health checks detect it and shift DNS to the DR region; the RDS read replica is promoted to a standalone writable primary, and Auto Scaling Groups in us-west-2 scale up to take full production load.

This design directly targets a low RTO (Recovery Time Objective) and low RPO (Recovery Point Objective) without paying for a full duplicate production environment running at 100% capacity around the clock.

2.4 Centralized Monitoring & Logging

- Amazon CloudWatch collects Metrics, Alarms, Dashboards, and Logs from both regions into one place.
- CloudWatch Logs / Log Insights lets the team query application and infrastructure logs quickly during an incident.
- CloudWatch Alarms trigger Notifications via Amazon SNS to Email/Slack, so the on-call engineer knows about a problem before the customer does.
- AWS CloudTrail records every API call for auditing and security investigation.

- Amazon EventBridge drives event-based automation — for example, triggering a Lambda function automatically when a backup completes or an alarm fires.

3. Architecture Diagram — Detailed VPC & Three-Tier Network Design

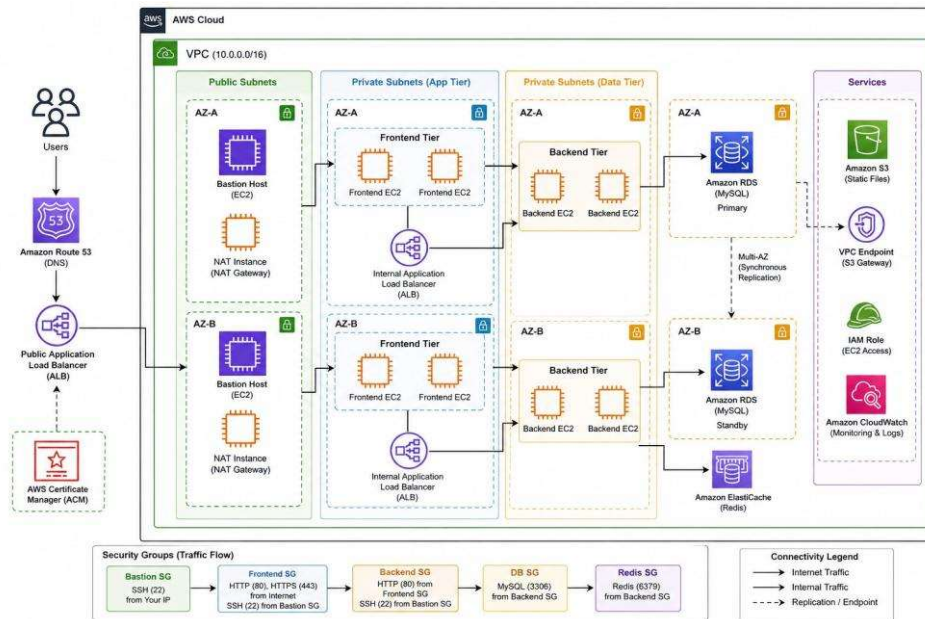


Figure 2: Single-region VPC (10.0.0.0/16) detailed subnet, security group, and tier-by-tier traffic flow.

3.1 Network Layout

- A single VPC (10.0.0.0/16) is divided into three logical layers, each duplicated across AZ-A and AZ-B for high availability: Public Subnets, Private Subnets (App Tier — containing both Frontend and Backend), and Private Subnets (Data Tier).
- Public Subnets host only the Bastion Host and NAT Instance/Gateway — nothing that serves application traffic directly sits in a public subnet.
- A public-facing Application Load Balancer (ALB), secured with an SSL/TLS certificate from AWS Certificate Manager (ACM), receives all inbound user traffic over HTTPS and forwards it into the private Frontend tier.

3.2 Tier-by-Tier Breakdown

Frontend Tier (Private Subnet — App Tier)

Two Frontend EC2 instances per AZ sit behind an Internal Application Load Balancer. They handle presentation logic and forward API calls to the backend.

Backend Tier (Private Subnet — App Tier)

Backend EC2 instances receive traffic only from the Frontend tier's internal ALB (never directly from the internet), enforcing a strict separation of concerns and reducing blast radius if the frontend is compromised.

Data Tier (Private Subnet — Data Tier)

Amazon RDS (MySQL) runs in Multi-AZ mode: a Primary instance in AZ-A synchronously replicates to a Standby in AZ-B, giving automatic failover with no data loss for committed transactions. Amazon ElastiCache (Redis) sits alongside RDS to cache frequent reads and session data, reducing database load and latency.

Services Layer

- Amazon S3 stores static files/assets, accessed privately through a VPC Endpoint (S3 Gateway) — traffic never leaves the AWS network to reach S3, improving both security and cost.
- IAM Roles are attached to EC2 instances so application code never hard-codes AWS credentials.
- Amazon CloudWatch provides monitoring and logs for this region's resources.

3.3 Security Group Chain (Defense in Depth)

Traffic is only allowed to move one tier at a time, and only from the specific security group in front of it — this is the core security control of the whole design:

- Bastion SG: SSH (22) allowed only from the engineer's own IP.
- Frontend SG: HTTP (80) / HTTPS (443) allowed from the internet (via the ALB); SSH (22) allowed only from the Bastion SG.
- Backend SG: HTTP (80) allowed only from the Frontend SG; SSH (22) allowed only from the Bastion SG.
- DB SG: MySQL (3306) allowed only from the Backend SG.
- Redis SG: Redis (6379) allowed only from the Backend SG.

No tier can be reached by skipping the one in front of it — even if an attacker reached the Frontend tier, they could not open a direct connection to the database; they would have to compromise the Backend tier first, and that tier itself cannot be reached from the public internet.

4. AWS Services Used — Quick Reference

AWS Service	Role in this Architecture
Amazon Route 53	DNS resolution and health-check-based failover routing between Primary and DR regions.
Amazon CloudFront	CDN that caches content close to users and serves as the active entry point in front of the ALB.
AWS WAF	Web Application Firewall protecting against common exploits (SQLi, XSS, bot traffic).
Elastic Load Balancing (ALB)	Distributes traffic across EC2 instances; public ALB for internet traffic, internal ALB between frontend and backend.
Amazon EC2 + Auto Scaling	Runs Web/App tier instances; scales in/out automatically based on load and replaces unhealthy instances.
Amazon RDS (MySQL)	Managed relational database with Multi-AZ synchronous replication and cross-region read replicas for DR.

Amazon ElastiCache (Redis)	In-memory caching layer to reduce database load and latency for frequent reads/session data.
Amazon S3	Stores static assets and backup artifacts, accessed privately via a VPC Gateway Endpoint.
AWS Backup	Centralized, scheduled backups with cross-region copy to a second Backup Vault for DR.
NAT Gateway	Lets private-subnet instances reach the internet (e.g., for patching) without being publicly reachable.
Bastion Host	Single hardened entry point for administrative SSH access into private subnets.
AWS Certificate Manager	Issues and manages the TLS certificate used to terminate HTTPS at the ALB.
Amazon CloudWatch	Central metrics, logs, dashboards, and alarms across both regions.
AWS CloudTrail	Audit log of every API call made in the account for security and compliance.
Amazon EventBridge	Event bus that triggers automated responses to operational events (e.g., alarm-triggered remediation).
Amazon SNS	Delivers alarm notifications to email/Slack for on-call response.
AWS IAM	Least-privilege roles and policies for EC2 instances and operators — no hard-coded credentials.

5. Interview Q&A — Anticipated Deep-Dive Questions

Below are the questions interviewers most commonly ask about a project like this, with concise, confident answers you can use as a script or adapt in your own words.

Q: Walk me through your architecture at a high level.

A: It's a three-tier web application — web, app, and database layers — deployed across two Availability Zones in a primary AWS region for high availability, with a warm-standby copy of the same environment in a second region for disaster recovery. Route 53 and CloudFront sit in front to route and cache traffic, WAF protects against common web exploits, and CloudWatch, X-Ray, and CloudTrail give me full visibility into performance, tracing, and audit activity across both regions.

Q: Why did you choose a three-tier architecture instead of a monolith or serverless design?

A: Three-tier gives clean separation of concerns — presentation, business logic, and data — which makes each layer independently scalable, testable, and securable. It's also the pattern most enterprises already run in production, so it demonstrates I can work with the infrastructure most teams actually have, while still being straightforward to evolve toward containers or serverless later if needed.

Q: How is your application highly available?

A: At the infrastructure level, every tier is duplicated across two Availability Zones behind Auto Scaling Groups, so losing one AZ, or even one instance, doesn't cause downtime. The database uses RDS Multi-AZ with synchronous

replication to a standby, so a database failover is automatic. Load balancers do continuous health checks and stop routing traffic to unhealthy targets within seconds.

Q: What's your disaster recovery strategy, and why warm standby instead of active-active or backup-and-restore?

A: I used a warm-standby approach: a scaled-down but fully deployed copy of the stack running in us-west-2, kept current through RDS cross-region read replicas and AWS Backup cross-region copies. Backup-and-restore alone would give too long a recovery time for a customer-facing app, while full active-active would double compute cost for a project that doesn't need that level of availability. Warm standby is the practical middle ground — I can promote the replica and scale up Auto Scaling Groups in minutes instead of hours.

Q: How does failover actually happen, step by step?

A: Route 53 health checks continuously probe the primary region's endpoint. If they fail past the configured threshold, Route 53 shifts DNS resolution to the DR region's CloudFront/ALB endpoint. At the same time, I promote the RDS cross-region read replica in us-west-2 to a standalone writable primary, and the DR region's Auto Scaling Groups scale from standby capacity up to production capacity. WAF, which is normally inactive in the DR path to save cost, is activated to protect the now-live traffic.

Q: How did you secure this environment?

A: Defense in depth through security groups: each tier only accepts traffic from the specific security group directly in front of it, so the database only hears from the backend, the backend only hears from the frontend load balancer, and nothing is reachable from the internet except the public ALB. Administrative access goes through a single bastion host restricted to my IP. I used ACM for TLS termination at the load balancer, IAM roles instead of hard-coded credentials on EC2, and a VPC endpoint so traffic to S3 never leaves the AWS network.

Q: How would you scale this to handle a 10x traffic spike?

A: Auto Scaling Groups on the web and app tiers already scale horizontally based on CPU/request-count metrics, so most of that is automatic. Beyond that I'd add read replicas or introduce ElastiCache more aggressively to take read load off RDS, consider RDS Proxy to pool database connections under high concurrency, and rely on CloudFront to absorb static content load so it never reaches the origin at all.

Q: What would you improve if you rebuilt this today?

A: I'd move the infrastructure definition into Infrastructure as Code — Terraform or CloudFormation — so both regions are provisioned from the same templates instead of manually kept in sync, which removes configuration drift as a failure mode. I'd also add automated failover testing (chaos-engineering style game days) so the DR path is actually exercised regularly, not just designed on paper.

Q: How do you monitor this system in production?

A: CloudWatch is the central hub — metrics and dashboards for both regions, alarms wired to SNS for email/Slack alerts so the team is notified before customers notice a problem. X-Ray gives distributed tracing to catch latency issues across the tiers, CloudTrail gives me a full audit trail of every API call for security review, and EventBridge lets me automate responses to specific events instead of relying purely on a human to react.

6. Full Project Explanation — Interview Narrative (Paragraph Form)

Use this as a single, flowing narrative you can say almost verbatim when an interviewer says 'walk me through this project' or 'tell me about a project you're proud of.' It's written to be spoken naturally in 2-3 minutes.

This project is a highly available, disaster-recovery-ready, three-tier web application that I designed and deployed entirely on AWS, and it's reachable at multicloud-devops.veerasir.. The goal was to build something that reflects how real production systems are actually run in industry — not just a single EC2 instance serving a website, but a properly layered architecture with redundancy, security boundaries, automated recovery, and full observability baked in from the start. I split the design into two views that work together: a network-level view that shows exactly how a request travels from a user to the database and back, and a business-continuity view that shows what happens when an entire AWS region becomes unavailable.

At the network level, I built a single VPC split into public and private subnets across two Availability Zones. The public subnets only contain a bastion host and a NAT gateway — nothing that serves customer traffic is ever directly internet-facing. All real traffic enters through a public Application Load Balancer secured with a TLS certificate from AWS Certificate Manager, and from there it's forwarded into a private frontend tier, then to a private backend tier behind an internal load balancer, and finally to the data tier. The data tier runs Amazon RDS for MySQL in Multi-AZ mode, so the database itself has synchronous replication to a standby and can fail over automatically without any data loss, and I added ElastiCache with Redis in front of it to absorb repeated read traffic and keep latency low. Every one of those hops is locked down with its own security group, so the database security group only accepts connections from the backend, the backend only accepts connections from the frontend load balancer, and administrative SSH access is only possible through the single hardened bastion host restricted to my own IP address. That means even if an attacker somehow reached the frontend tier, there is no direct path from there to the database — they would have to compromise an entire additional tier first, and that tier isn't reachable from the internet at all.

The second half of the project is the disaster recovery story, because high availability inside one region isn't enough for a production system — regions do occasionally have outages. So I deployed a second, warm-standby copy of the same three-tier stack in us-west-2, while the primary lives in us-east-1. Amazon Route 53 sits at the very front with health checks constantly monitoring the primary region; under normal conditions it routes users through Amazon CloudFront to the active primary environment, while AWS WAF stays provisioned but inactive on the DR path to keep costs down until it's actually needed. Behind the scenes, Amazon RDS continuously replicates from the primary database to a cross-region read replica in the DR region, and AWS Backup takes scheduled snapshots into a backup vault that gets copied cross-region as well, so I have both a live, nearly-real-time replica and independent point-in-time backups. If the primary region goes down, Route 53's health checks detect the failure and shift DNS to the DR region, I promote the read replica to a standalone writable database, and the DR region's Auto Scaling Groups scale up from standby capacity to full production capacity — all without anyone needing to manually rebuild infrastructure under pressure during an incident.

Finally, I treated observability as a first-class part of the architecture rather than an afterthought. Amazon CloudWatch centralizes metrics, logs, dashboards, and alarms from both regions, and any alarm that fires sends a notification through Amazon SNS to email and Slack, so the team knows about a problem before a customer has

to report it. AWS X-Ray gives me distributed tracing across the web and app tiers to pinpoint exactly where latency is coming from, AWS CloudTrail records every single API call made in the account for security auditing, and Amazon EventBridge lets me wire automated responses to specific operational events instead of relying purely on a human being to react in real time. Put all together, this project demonstrates the full lifecycle of running a serious application on AWS: secure network design, high availability within a region, automated disaster recovery across regions, and the monitoring and alerting needed to actually operate it with confidence in production — which is exactly the kind of system design and operational thinking I'd bring to this role.

7. Presentation Tips for the Interview

- Start with the one-sentence summary first (what it is and where it's hosted), then go top-down: DNS → CDN/WAF → Load Balancer → Web → App → DB, before diving into DR.
- If asked to draw it on a whiteboard, draw Diagram 2 (the detailed VPC) first — it's easier to sketch live — then layer the DR region on top as a second box connected by replication arrows.
- Always tie a technical choice back to a business reason: Multi-AZ RDS isn't just 'a feature,' it's 'so a database failure doesn't cause an outage.' Interviewers remember the 'why' more than the acronym.
- Have one honest limitation ready (e.g., manual infrastructure provisioning instead of IaC, or DR failover not yet automated end-to-end) — it shows self-awareness and gives you a natural 'what I'd improve' answer.